

Scapegoat Tree - частично сбалансированное дерево поиска (степень сбалансированности может быть настроена), такое что операции поиска работает за $O(\log(n))$, вставки и удаления работают амортизированно за $O(\log(n))$, при этом скорость одной операции может быть улучшена в ущерб другой (с помощью выбора α).

1 Определения

Пусть $T = (V, E)$ - дерево, $x \in V$:

$size(T) = |V|$

$d(x)$ - глубина x (расстояние от корня до x)

$size(x)$ - размер поддерева вершины x

$h(x)$ - высота поддерева x

$h(T)$ - высота дерева

$h_\alpha(n) := \lfloor \log_{\frac{1}{\alpha}}(n) \rfloor$, $h_\alpha(T) := h_\alpha(|V|)$

Зафиксируем $\frac{1}{2} < \alpha < 1$.

def: Вершина x - α -весо-сбалансированная, если

$$size(x.left) \leq \alpha \cdot size(x) \text{ и } size(x.right) \leq \alpha \cdot size(x).$$

def: Дерево T - α -весо-сбалансированное, если все его вершины α -весо-сбалансированные.

def: Дерево T - α -высото-сбалансированное, если $h(T) \leq h_\alpha(T)$.

def: Дерево T - почти- α -высото-сбалансированное, если $h(T) \leq h_\alpha(T) + 1$.

def: Вершина x - глубокая, если $d(x) > h_\alpha(T)$.

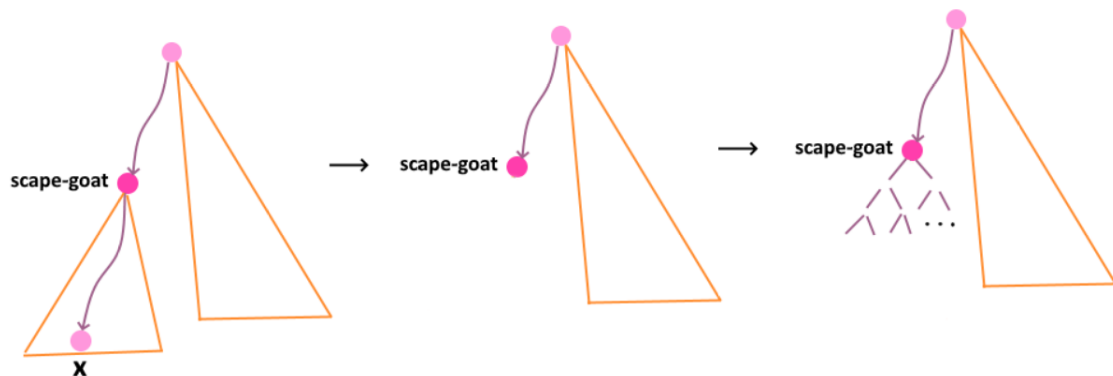
Легко проверить, что весо-сбалансированное дерево является высото-сбалансированным.

2 Операции

♡ Поиск - как в бинарном дереве поиска.

♡ Вставка - как в бинарном дереве поиска. Если после вставки вершина, которую мы вставили оказалась глубокой, то происходит перебалансировка.

- ♡ Перебалансировка - если вставленная вершина x оказалась глубокой, идем от x к корню и ищем на этом пути самого глубокого не-весобалансированного предка x (такая вершина называется *scapegoat* и она всегда найдётся). Если мы нашли *scape* – *goat* вершину, полностью перестраиваем ее поддерево в сбалансированное бинарное ($\frac{1}{2}$ -весобалансированное) дерево поиска и завершаем работу. Такое перестраивание можно сделать за линейное время: обходим дерево слева направо, кладем элементы в массив. Корнем делаем центральный элемент, делим массив на 2 части, рекурсивно вызываемся от половинок



- ♡ Удаление:

В удалении же у нас тоже будет происходить перестройка, но теперь всего дерева, но это будет происходить не всегда.

Пусть $maxSize(T)$ - максимальный размер дерева с момента его перестройки при удалении (именно при удалении). Удалим вершину x как в бинарном дереве поиска. Если $size(T) < \alpha \cdot maxSize(T)$, то полностью перестраиваем дерево в сбалансированное бинарное дерево поиска и обновляем $maxSize(T) = size(T)$.

3 Корректность

theorem: Scape-goat вершина на пути от глубокой до корня обязательно существует.

Это делает перебалансировку возможной.

theorem: Если поддерживать только операции вставки и поиска, то дерево всегда высотобалансированное.

theorem: При операциях удаления и вставки дерево всегда почти-высотобалансированное.

4 Асимптотика

- ◇ Поиск - так как дерево всегда почти-высото-сбалансированное, то высота его всегда $O(\log(\text{size}(T)))$, а значит поиск работает за $O(\log(\text{size}(T)))$.
- ◇ Вставка - покажем с помощью метода потенциалов, что амортизированно работает за $O(\log(\text{size}(T)))$:

Пусть x вершина. Обозначим:

$$\Delta x := |\text{size}(x.\text{left}) - \text{size}(x.\text{right})|$$

$$\Phi(x) := \begin{cases} 0 & \text{если } \Delta x < 2 \\ \Delta x & \text{иначе} \end{cases}$$

Потенциал дерева = сумме потенциалов вершин.

Заметим, что потенциал $\frac{1}{2}$ -весов-сбалансированного дерева = 0.

Потенциал перед перебалансировкой:

Пусть x не является α -весов-сбалансированной, тогда без ограничения общности

$$\text{size}(x.\text{left}) > \alpha \text{size}(x)$$

Возьмём $\varepsilon > 0$ очень маленькое, что всё ещё:

$$\text{size}(x.\text{left}) > (\alpha + \frac{\varepsilon}{2}) \text{size}(x)$$

Тогда

$$\begin{aligned} \Delta x &:= \text{size}(x.\text{left}) - \text{size}(x.\text{right}) = \text{size}(x.\text{left}) - (\text{size}(x) - \text{size}(x.\text{left}) - 1) = \\ &= 2\text{size}(x.\text{left}) - \text{size}(x) > (2\alpha + \varepsilon - 1)\text{size}(x) \geq \varepsilon \cdot \text{size}(x) \end{aligned}$$

Таким образом потенциал *scape* – *goat* вершины перед перебалансировкой:

$$\Phi(x) = \Theta(\text{size}(x))$$

а после = 0.

Заметим, что на поиск *scape* – *goat* вершины мы тратим $O(\text{size}(x))$ (каждую вершину мы спросим о ее размере один раз), на перебалансировку также уходит $O(\text{size}(x))$. Оплатим поиск *scape* – *goat* вершины и перебалансировку этим потенциалом - получаем амортизированно $O(1)$ на операцию. Остальные операции выполняются за $O(\log(\text{size}(T)))$ (вставка как в двоичном дереве поиска).

- ◇ Удаление - амортизированно работает за $O(\log(\text{size}(T)))$ (метод амортизации - бухгалтерский учёт):

Перестройка происходит редко: чтобы перестройка наступила, нужно

$$size(T) < \alpha \cdot maxSize(T).$$

Сколько удалений для этого нужно? Каждое удаление уменьшает размер на 1. После перестройки: $size = maxSize$, а значит размер должен уменьшиться с $maxSize(T)$ до $\alpha \cdot maxSize(T)$. Для этого понадобится

$$maxSize(T) - \alpha \cdot maxSize(T) = (1 - \alpha)maxSize(T)$$

удалений.

Давайте заплатим заранее за будущую перестройку. При перестройке тратим $O(maxSize(T))$ времени. Разобьём эту стоимость на предыдущие $(1 - \alpha)maxSize(T)$ удалений, которые привели к этой перестройке. Каждое из этих удалений заплатит «монетками» вперёд. Платим за одно удаление:

$$\frac{maxSize(T)}{(1 - \alpha)maxSize(T)} = \frac{1}{1 - \alpha} = O(1)$$

Это означает: если каждое удаление (кроме обычной своей работы) откладывает константу монет, то к моменту следующей перестройки этих монет хватит, чтобы оплатить её.